



Coinbase: Nitro Enclave TEE Security Review

Cantina Managed review by:
Zigtur, Lead Security Researcher
0xHuy0512, Security Researcher

March 30, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Deregistering a PCR0 does not invalidate signers registered under that PCR0	4
3.2	Informational	5
3.2.1	registerPCR0 has no length validation on the PCR0 input	5
3.2.2	Error NotImplemented() is declared but never used	5
3.2.3	EnumerableSet implementation inconsistency across contracts	5
3.2.4	keccak256 length parameter in assembly could be hardcoded in registerSigner	6
3.2.5	AWS Nitro PCR0-only check has a theoretical weakness	6
3.2.6	Attestation can be valid up to 1 hour after certificate expiration	7
3.2.7	Wrong event emitted when setting a route verifier to the frozen sentinel address	7
3.2.8	Truncation in millisecond-to-second conversion allows near-future attestation timestamps to bypass validation	7
3.2.9	Inconsistent staleness comparison between TEEProverRegistry and NitroEnclaveVerifier rejects boundary-valid attestations	8
3.2.10	Redundant storage of verifier/aggregator ID sets and proof ID mapping	8

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Mar 19th to Mar 23rd the Cantina team conducted a review of [contracts](#) on commit hash [2421afdd](#). The team identified a total of **11** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	10	9	1
Total	11	10	1

2.1 Scope

The security review had the following components in scope for [contracts](#) on commit hash [2421afdd](#):

```
src/multiproof/tee
├─ NitroEnclaveVerifier.sol
├─ TEEProverRegistry.sol
└─ TEEVerifier.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Deregistering a PCR0 does not invalidate signers registered under that PCR0

Severity: Medium Risk

Context: TEEVerifier.sol#L83-L95

Description: When `TEEProverRegistry.deregisterPCR0()` is called, it only removes the PCR0 from the `validPCR0s` whitelist. It does not invalidate any signers that were previously registered with that PCR0. Then, `TEEVerifier.verify()` checks `signerPCR0[signer]` which remains set, and never checks whether that PCR0 is still in `validPCR0s`.

```
// TEEProverRegistry.sol:102-108
function deregisterPCR0(bytes calldata pcr0) external onlyOwner {
    bytes32 pcr0Hash = keccak256(pcr0);
    delete validPCR0s[pcr0Hash];
    emit PCR0Deregistered(pcr0Hash);
}

// TEEVerifier.sol:83-95
function verify(
    bytes calldata proofBytes,
    bytes32 imageId,
    bytes32 journal
)
// ...
{
    // ...

    // Get the PCR0 the signer was registered with
    bytes32 registeredPCR0 = TEE_PROVER_REGISTRY.signerPCR0(signer);

    // Check that the signer is registered (PCR0 != 0)
    if (registeredPCR0 == bytes32(0)) {
        revert InvalidSigner(signer);
    }

    // @audit never checks validPCR0s[registeredPCR0]

    // Check that the signer's registered PCR0 matches the claimed imageId
    // This ensures the signer was running the exact enclave image specified
    if (registeredPCR0 != imageId) {
        revert ImageIdMismatch(registeredPCR0, imageId);
    }
}
```

This means if a PCR0 is deregistered (e.g., because the enclave image is compromised), all previously registered signers under that PCR0 remain fully operational. This leads to multiple edge-cases:

- If a PCR0 is registered, deregistered, and registered again, signers from the first registration remain active.
- `getRegisteredSigners()` returns signers associated with an invalid PCR0.

Recommendation: A way to fix this issue is to add a call to `TEEProverRegistry.validPCR0s()` to ensure that the PCR0 attached to the signer is still valid.

Alternatively, an `isValidSigner` function on `TEEProverRegistry` that includes the `validPCR0s` check and is used from `TEEVerifier` could be implemented.

Coinbase: Fixed in commit [a94f57a](#). A signer's PCR0 is now removed in `deregisterSigner()`.

Cantina Managed: Fixed. A check that the correct image hash is used has also been added in `TEEVerifier.verify`.

3.2 Informational

3.2.1 registerPCR0 has no length validation on the PCR0 input

Severity: Informational

Context: TEEProverRegistry.sol#L96

Description: TEEProverRegistry.registerPCR0 accepts arbitrary-length bytes calldata pcr0 and hashes it with keccak256. However, AWS Nitro PCR0 values are SHA-384 hashes (48 bytes). There is no check that the input is 48 bytes.

```
// TEEProverRegistry.sol:96-99
function registerPCR0(bytes calldata pcr0) external onlyOwner {
    bytes32 pcr0Hash = keccak256(pcr0);
    validPCR0s[pcr0Hash] = true;
    emit PCR0Registered(pcr0Hash);
}
```

Recommendation: Add a length check on pcr0.

```
if (pcr0.length != 48) revert InvalidPCR0Length();
```

Coinbase: Fixed in commit a94f57a.

Cantina Managed: Fixed. The registerPCR0 function has been removed, the direct image hash (i.e. the hash of the PCR0) is retrieved from the AggregateVerifier. Moreover, _extractPCR0Hash ensures that the PCR0 is 48 bytes.

3.2.2 Error NotImplemented() is declared but never used

Severity: Informational

Context: NitroEnclaveVerifier.sol#L119-L121

Description: The NotImplemented error is defined but it is never used in the codebase.

```
// NitroEnclaveVerifier.sol:119-120
/// @dev Thrown when calling verifyWithProgramId or batchVerifyWithProgramId, which are
↳ intentionally disabled
error NotImplemented();
```

Recommendation: Remove the unused error.

Coinbase: Fixed in commit 7ee57f6.

Cantina Managed: Fixed. The error implementation has been removed.

3.2.3 EnumerableSet implementation inconsistency across contracts

Severity: Informational

Context: NitroEnclaveVerifier.sol#L5, TEEProverRegistry.sol#L14

Description: NitroEnclaveVerifier uses OpenZeppelin EnumerableSet, while TEEProverRegistry uses Solady EnumerableSetLib. These are different implementations but offer the same functionalities.

```
// NitroEnclaveVerifier.sol
import { EnumerableSet } from "@openzeppelin/contracts/utils/structs/EnumerableSet.sol";

// TEEProverRegistry.sol
import { EnumerableSetLib } from "@solady-v0.0.245/utils/EnumerableSetLib.sol";
```

Recommendation: Use the same EnumerableSet implementation in both contracts.

Coinbase: Fixed in commit 567a3a0 by removing the use of EnumerableSet in NitroEnclaveVerifier.

Cantina Managed: Fixed. Only one EnumerableSet implementation is used in the review scope.

3.2.4 keccak256 length parameter in assembly could be hardcoded in registerSigner

Severity: Informational

Context: TEEProverRegistry.sol#L130-L134

Description: In TEEProverRegistry.registerSigner, the keccak256 call in assembly uses sub(mload(pubKey), 1) for the length parameter. Since pubKey.length == 65 is already enforced, this could be hardcoded to 64.

```
// TEEProverRegistry.sol:130-134
if (pubKey.length != 65) revert InvalidPublicKey();
bytes32 publicKeyHash;
assembly {
    publicKeyHash := keccak256(add(pubKey, 0x21), sub(mload(pubKey), 1)) // could be
    ↪ 64
}
```

Recommendation: Use 64 as the length parameter to the keccak opcode.

```
diff --git a/src/multiproof/tee/TEEProverRegistry.sol
↪ b/src/multiproof/tee/TEEProverRegistry.sol
index 616bb514..e2cd2006 100644
- -- a/src/multiproof/tee/TEEProverRegistry.sol
+ ++ b/src/multiproof/tee/TEEProverRegistry.sol
@@ -130,7 +130,8 @@ contract TEEProverRegistry is OwnableManagedUpgradeable, ISemver {
    if (pubKey.length != 65) revert InvalidPublicKey();
    bytes32 publicKeyHash;
    assembly {
-         publicKeyHash := keccak256(add(pubKey, 0x21), sub(mload(pubKey), 1))
+         // Length is hardcoded to 64 to skip the 0x04 prefix and hash only the x
↪ and y coordinates
+         publicKeyHash := keccak256(add(pubKey, 0x21), 64)
    }
    address enclaveAddress = address(uint160(uint256(publicKeyHash)));
```

Coinbase: Fixed in commit f88f8d0.

Cantina Managed: Fixed. Recommended patch has been applied.

3.2.5 AWS Nitro PCR0-only check has a theoretical weakness

Severity: Informational

Context: TEEProverRegistry.sol#L124-L125

Description: TEEProverRegistry only validates PCR0 from the attestation. AWS Nitro PCRs 0 through 2 are SHA-384 hashes over sections' data without domain separation - sections are simply concatenated and headers are not included:

```
PCR-0: sha384('\0'*48 | sha384(Kernel | Cmdline | Ramdisk[:]))
PCR-1: sha384('\0'*48 | sha384(Kernel | Cmdline | Ramdisk[0]))
PCR-2: sha384('\0'*48 | sha384(Ramdisk[1:]))
```

Because there is no domain separation, bytes can theoretically be moved between adjacent sections without changing the PCR. For example, stripping bytes from the beginning of the second ramdisk and appending them to the first one would not change PCR-0.

This has never been demonstrated as exploitable in practice, particularly due to the cpio archive format's internal structure.

Recommendation: For defense in depth, validate PCR0, PCR1, and PCR2 together. In practice, checking PCR0 alone is likely safe enough, especially since registerPCR0 is owner-gated.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.6 Attestation can be valid up to 1 hour after certificate expiration

Severity: Informational

Context: [TEEProverRegistry.sol#L122](#)

Description: The ZK prover verifies that the certificate chain is valid at the attestation timestamp (`doc.timestamp / 1000`). `TEEProverRegistry` then allows attestations up to `MAX_AGE = 60` minutes old. This means a certificate that expires at time `T` can still produce valid attestations until `T + 60` minutes, because:

1. An attestation generated at time `T - 1` (just before expiry) has a valid cert chain.
2. This attestation can be submitted to the contract up to `T + 59min59s`.

In practice, this is acceptable since certificate private keys are unlikely to be compromised within 1 hour of expiration.

Recommendation: This edge-case should be documented.

Coinbase: Fixed in commit [7acbf62](#) by documenting the edge-case.

Cantina Managed: Fixed.

3.2.7 Wrong event emitted when setting a route verifier to the frozen sentinel address

Severity: Informational

Context: [NitroEnclaveVerifier.sol#L477-L486](#)

Description: In `addVerifyRoute()`, the function checks whether the *current* route verifier is `FROZEN` (`address(0xdead)`) and reverts if so. However, it does not validate whether the incoming `_verifier` parameter itself equals `FROZEN`. An owner can therefore call `addVerifyRoute()` with `_verifier = address(0xdead)`, which sets the route to the frozen sentinel value while emitting `ZkRouteAdded` instead of `ZkRouteWasFrozen`. This bypasses the intended `freezeVerifyRoute()` flow and emits a misleading event, causing off-chain systems tracking route freezes to miss this state change.

Recommendation: Add a check that `_verifier` is not `FROZEN` in `addVerifyRoute()` function.

Coinbase: Fixed in commit [4719a37](#).

Cantina Managed: Fixed, a check has been added and reverts with the new `InvalidVerifierAddress` error.

3.2.8 Truncation in millisecond-to-second conversion allows near-future attestation timestamps to bypass validation

Severity: Informational

Context: [NitroEnclaveVerifier.sol#L657-L661](#)

Description: In `NitroEnclaveVerifier` contract, the `_verifyAttestation()` function converts the journal's millisecond timestamp to seconds via `journal.timestamp / 1000`. This integer division truncates up to 999ms. The subsequent future-time check `timestamp > block.timestamp` therefore permits attestations timestamped up to ~999ms in the future to pass validation. While a future-dated attestation is unlikely in practice, if one does occur it signals something is wrong, so a stricter check is warranted.

Recommendation: Use `>=` instead of `>` for the future-time bound. Combining with finding "Inconsistent staleness comparison between `TEEProverRegistry` and `NitroEnclaveVerifier` rejects boundary-valid attestations", it should be fixed to:

```
- if (timestamp + maxTimeDiff < block.timestamp || timestamp > block.timestamp) {  
+ if (timestamp + maxTimeDiff <= block.timestamp || timestamp >= block.timestamp) {
```

Coinbase: Fixed in commit [37edf92](#).

Cantina Managed: Fixed. Recommended patch has been applied.

3.2.9 Inconsistent staleness comparison between TEEProverRegistry and NitroEnclaveVerifier rejects boundary-valid attestations

Severity: Informational

Context: NitroEnclaveVerifier.sol#L658, TEEProverRegistry.sol#L122

Description: TEEProverRegistry.registerSigner() and NitroEnclaveVerifier._verifyJournal() both perform attestation staleness checks, but use different comparison operators. TEEProverRegistry uses `<= (journal.timestamp / MS_PER_SECOND + MAX_AGE <= block.timestamp)`, while NitroEnclaveVerifier uses `< (timestamp + maxTimeDiff < block.timestamp)`. An attestation at exactly MAX_AGE seconds old passes NitroEnclaveVerifier.verify() successfully but is then rejected by TEEProverRegistry.registerSigner() with AttestationTooOld().

Recommendation: Align both contracts to use the same comparison operator. Change NitroEnclaveVerifier._verifyJournal() to use `<=`. Combining with issue #9, it should be fixed to:

```
- if (timestamp + maxTimeDiff < block.timestamp || timestamp > block.timestamp) {  
+ if (timestamp + maxTimeDiff <= block.timestamp || timestamp >= block.timestamp) {
```

Coinbase: Fixed in commit 37edf92.

Cantina Managed: Fixed. Recommended patch has been applied.

3.2.10 Redundant storage of verifier/aggregator ID sets and proof ID mapping

Severity: Informational

Context: NitroEnclaveVerifier.sol#L54-L76

Description: NitroEnclaveVerifier maintains _verifierIdSet, _aggregatorIdSet, and _verifierProofIds mappings alongside zkConfig. In practice, only the latest verifierId and aggregatorId stored in zkConfig are used during verify() and batchVerify(). The _verifierProofIds mapping is only ever queried in batchVerify() with the current zkConfig verifierId. The redundant state adds unnecessary gas overhead to setZkConfiguration(), updateVerifierId(), and updateAggregatorId(), and introduces functions (removeVerifierId(), removeAggregatorId()) that operate on unused data structures.

Recommendation: Remove _verifierIdSet, _aggregatorIdSet, _verifierProofIds, the EnumerableSet dependency, and the removeVerifierId()/removeAggregatorId() functions. Move verifierProofId into ZkCoProcessorConfig as a single bytes32 field.

Coinbase: Fixed in commit 567a3a0 and PR 226 commit 365dc57.

Cantina Managed: Fixed. The _verifierIdSet and _aggregatorIdSet mappings have been removed. _verifierProofIds has been simplified to a single mapping.